컴포넌트마다 반복되는 세 줄

TokenZswap 프론트엔드 · `frontend/src/App.tsx`, `main.tsx`, `contracts.ts` 기준

```
const connection  = useConnection()                    // ① 지갑 연결 상태
const addresses   = ADDRESSES[connection.chainId ?? 0]  // ② 이 체인의 컨트랙트 주소
const queryClient = useQueryClient()                   // ③ 공유 캐시 관리자
```

`WalletBar`, `PoolInfo`, `MyBalance`, `Swap`, `AddLiquidity`, `RemoveLiquidity`, `MyNFTs`. 일곱 컴포넌트가 모두 이 줄들로 시작해요. 같은 일을 일곱 번 하는 것처럼 보이지만 실제로는 그렇지 않아요.

> **한 줄 요약.** 객체를 **만드는** 곳은 `main.tsx` 한 곳뿐이에요. 각 컴포넌트는 이미 만들어진 객체를 **꺼내 쓰고**, 값이 바뀌면 알려 달라고 **구독**해요. 꺼내는 코드는 그 값이 필요한 컴포넌트 안에 있어야 해서, 컴포넌트마다 반복되는 거예요.

1 큰 그림: 만드는 곳 하나, 꺼내는 곳 여럿

```
// main.tsx: 브라우저가 앱을 열 때 딱 한 번 실행
const queryClient = new QueryClient()              // ← 캐시 관리자를 실제로 "생성"

<WagmiProvider config={config}>                   // ← 지갑 연결 상태를 아래 전체에 공급
  <QueryClientProvider client={queryClient}>     // ← queryClient를 아래 전체에 공급
    <App />                                       //    App 안의 컴포넌트 7개가 전부 이 안에 있음
  </QueryClientProvider>
</WagmiProvider>
```

`Provider`는 React의 **Context** 기능이에요. 트리 위쪽에 값을 올려 두면, 그 아래 어떤 컴포넌트든 props로 받지 않고도 직접 꺼낼 수 있어요. `useConnection()`과 `useQueryClient()`가 바로 그 "꺼내는" 함수예요.

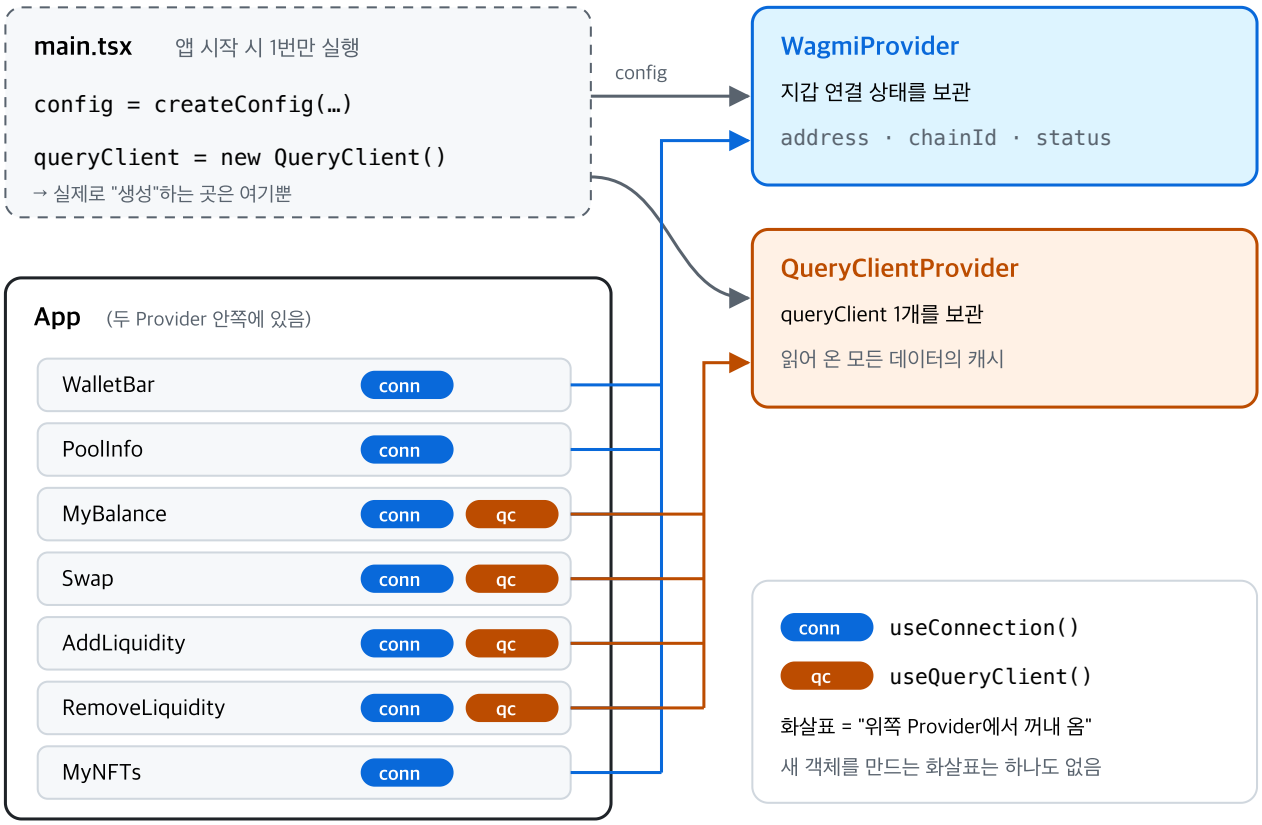


그림 1. 생성은 main.tsx에서 한 번. 컴포넌트 7개는 모두 같은 Provider에서 값을 꺼내 쓴다.

C로 비유하면 `main()`에서 전역 구조체를 한 번 초기화해 두고, 각 함수가 `get_ctx()`로 **같은 포인터**를 받아 쓰는 구조예요. 함수마다 `malloc`을 다시 하는 게 아니에요.

2 그럼 왜 파일 맨 위에 한 번만 쓰면 안 될까?

이유 1. 컴포넌트는 서로 독립된 함수예요

`Swap`과 `PoolInfo`는 별개의 함수라서, 한쪽의 지역 변수를 다른 쪽이 볼 수 없어요. 값이 필요한 함수가 각자 꺼내야 합니다.

이유 2. 훅은 "렌더링 중인 컴포넌트" 안에서만 동작해요

✗ 에러

```
const c = useConnection()  // 컴포넌트 밖

function Swap() {
  const me = c.address
}
```

✓ 정상

```
function Swap() {
  const c = useConnection()  // 안
  const me = c.address
}
```

React는 컴포넌트를 렌더링할 때 "지금 누구를 그리는 중인지" 기억해 두고, 그 컴포넌트에 딸린 **훅 목록**에 호출 결과를 차례로 적어 둬요. 파일 맨 위 코드는 앱이 로드될 때 실행돼서 그리는 중인 컴포넌트가 없어요. 그래서 기록할 자리가 없고, `Invalid hook call` 에러가 납니다.

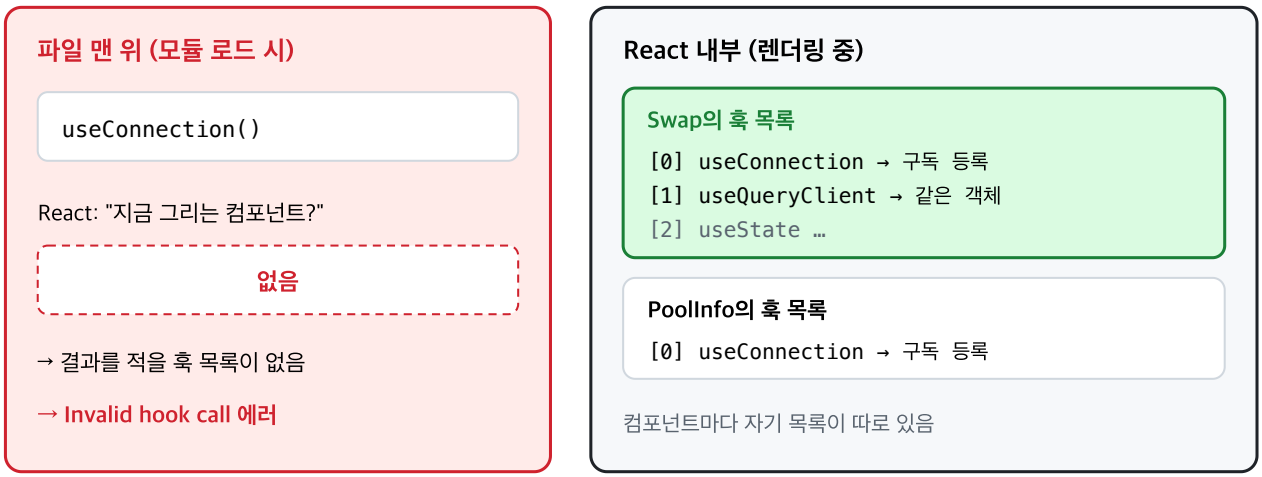


그림 2. 훅은 "지금 렌더링 중인 컴포넌트"의 목록에 기록된다. 그래서 컴포넌트 안에서만 호출할 수 있다.

이유 3. 값이 바뀌면 그 컴포넌트를 다시 그려야 해요

그림 2의 "구독 등록"이 핵심이에요. 컴포넌트가 `useConnection()`을 호출하면, React는 "이 컴포넌트는 연결 상태가 바뀌면 다시 그려야 한다"고 기록해요. 자세한 동작은 다음 절의 그림 3에서 볼게요.

3 ① `useConnection()`: 지갑 상태 읽기 + 구독

wagmi 훅이에요(예전 이름은 `useAccount`). `WagmiProvider`가 보관한 연결 상태를 돌려줘요.

```
connection.address      // '0xf39F…' 또는 undefined (지갑 미연결)
connection.chainId      // 31337 / 97 / 11155111 또는 undefined
connection.isConnected  // true / false
```

값을 한 번 읽고 끝나는 게 아니라 **구독**도 함께 해요. MetaMask에서 계정이나 네트워크를 바꾸면 wagmi 상태가 바뀌고, 구독한 컴포넌트만 골라서 다시 렌더링돼요.

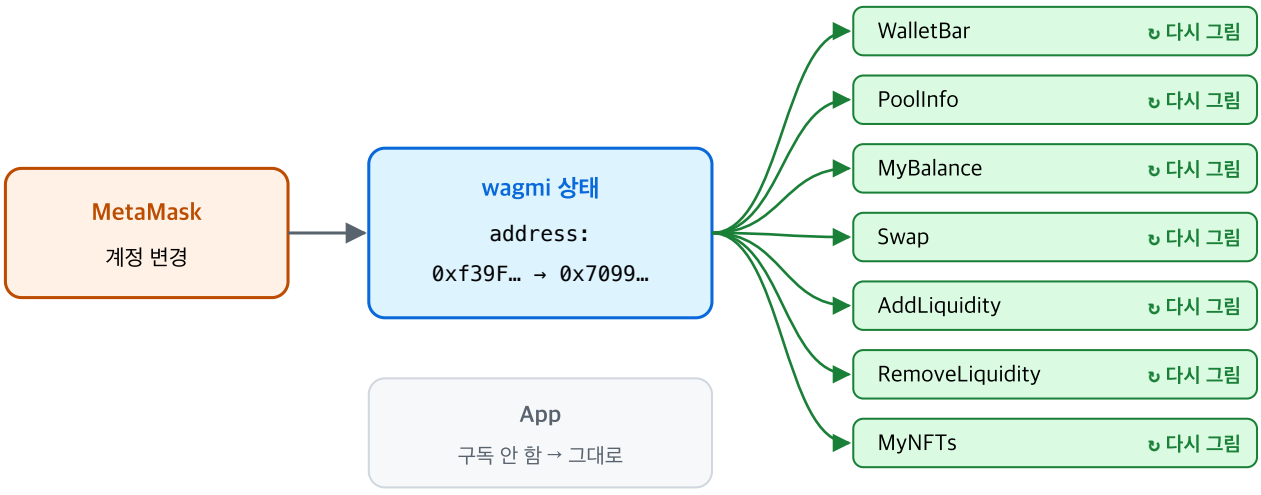


그림 3. 상태가 바뀌면 `useConnection()`을 호출한 컴포넌트만 다시 렌더링된다. 다시 그릴 때 세 줄이 또 실행되면서 새 값을 받는다.

> "매번 호출된다"는 건 **렌더링될 때마다 이 줄이 다시 실행된다**는 뜻이에요. 그때마다 연결이 새로 생기는 게 아니라, wagmi 상태에 있는 **최신 값**을 다시 읽어 오는 거예요. 그래서 계정을 바꾸면 `me`가 자동으로 새 주소가 됩니다.

4 ② `ADDRESSES[connection.chainId ?? 0]`: 체인별 주소 고르기

이 줄은 훅이 아니라 **평범한 객체 조회**예요. `contracts.ts`의 `ADDRESSES`는 체인 ID를 키로 하는 표예요. 지금은 로컬 anvil(`31337`) 한 칸만 채워져 있어요.

```
export const ADDRESSES: Record<number, Deployment> = {
  31337: { amm: '0x9fE4…', tokenA: '0x5FbD…', tokenB: '0xe7f1…', nft: '0xCf7E…' },
}
```

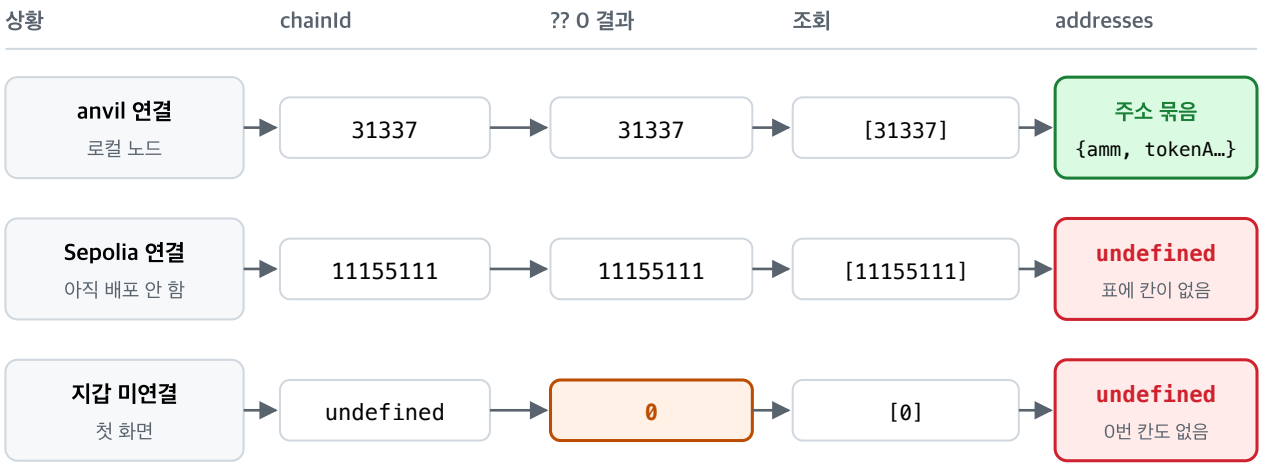


그림 4. "미연결"과 "지원 안 하는 체인"이 모두 `undefined` 하나로 모인다. Sepolia는 `config.ts`의 체인 목록에는 있지만 아직 배포 주소가 없다.

- `?? 0`: `chainId`가 `undefined`면 `0`을 대신 써요. 객체 키로 `undefined`를 쓰지 않기 위해서예요.
- 그 결과 아래 코드는 **`addresses`가 있는지 하나만** 확인하면 돼요.

```
args: me && addresses ? [me, addresses.amm] : undefined   // 둘 다 있을 때만 인자 구성
query: { enabled: !!addresses && !!me }                   // 둘 다 있을 때만 RPC 요청
if (!addresses || !me) return null                        // 없으면 화면을 안 그림
```

> **타입 함정.** `Record<number, Deployment>`는 "어떤 숫자 키든 값이 있다"는 뜻이라, TypeScript는 `addresses`를 `undefined`가 될 수 없는 타입으로 봐요. 실제로는 그림 4처럼 `undefined`가 나와요. 지금 코드는 `addresses?.amm`, `!!addresses`로 직접 확인하고 있어서 안전해요. 타입까지 정확하게 맞추려면 `Record<number, Deployment | undefined>`로 바꾸면 돼요.

5 ③ `useQueryClient()` : 모두가 같은 캐시 관리자를 받음

`main.tsx`에서 만든 **그 하나의** **`queryClient`**를 돌려줘요. `Swap`에서 받든 `MyBalance`에서 받든 메모리상 **같은 객체**예요. 이 객체가 `useReadContract`, `useReadContracts`, `useQuery`로 읽어 온 모든 데이터의 캐시를 관리해요.

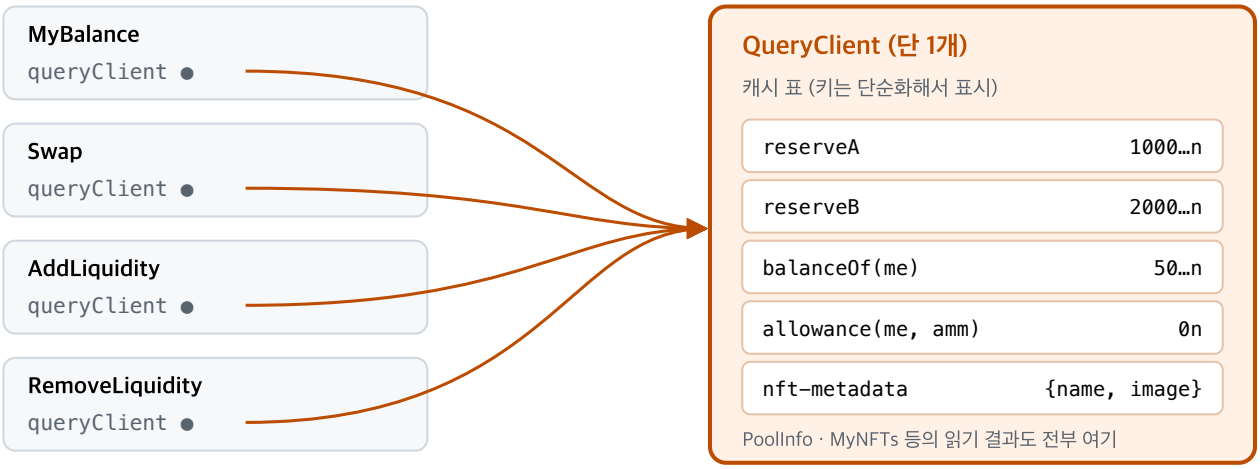


그림 5. 네 컴포넌트의 `queryClient` 변수는 모두 같은 객체를 가리킨다(C로 치면 같은 포인터).

왜 같은 객체여야 할까? 트랜잭션 후 새로고침

쓰기 기능이 있는 컴포넌트들은 모두 이 코드를 가지고 있어요.

```
useEffect(() => {
  if (receipt.isSuccess) queryClient.invalidateQueries()   // "캐시가 전부 낡았으니 다시 읽어라"
}, [receipt.isSuccess, queryClient])
```

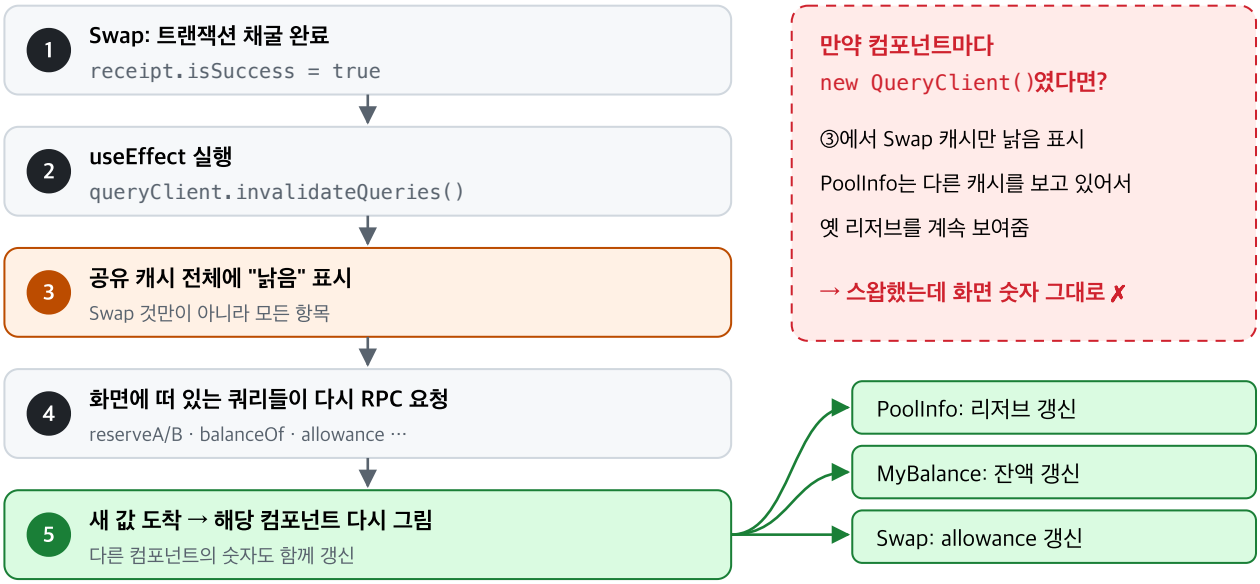


그림 6. 캐시가 하나로 공유되므로, Swap에서 한 번 무효화하면 다른 컴포넌트의 숫자까지 함께 새로고침된다.

6 C로 다시 보기

```c
/* main.c: 시작할 때 한 번 (= main.tsx) */
static WalletState  g_wallet;          // WagmiProvider가 보관하는 것
static QueryClient  g_qc;              // QueryClientProvider가 보관하는 것

int main(void) { wallet_init(&g_wallet); qc_init(&g_qc); run_app(); }

const WalletState *use_connection(void) {
    subscribe(current_component);  // 이 컴포넌트를 구독자로 등록
    return &g_wallet;
}
QueryClient *use_query_client(void) { return &g_qc; }  // 항상 같은 주소

/* 각 컴포넌트 (= Swap, PoolInfo …) */
void render_swap(void) {
    const WalletState *conn = use_connection();          // ① 꺼내기 + 구독
    const Deployment  *addr = lookup(conn->chain_id);     // ② 표 조회 (없으면 NULL)
    QueryClient       *qc   = use_query_client();        // ③ 같은 포인터
    if (!addr || !conn->address) return;
    /* … */
}
```

`render_swap`이 몇 번 호출되든 `g_wallet`과 `g_qc`는 하나예요. 반복되는 건 "포인터를 받아 오는 코드"뿐이에요.

7 정리

줄	종류	매번 새로 만드나?	하는 일
`useConnection()`	wagmi 훅	아니요. 공유 상태를 읽음	지갑 상태 읽기 + 변경 구독 (그림 3)
`ADDRESSES[…]`	일반 객체 조회	매번 계산하지만 표에서 한 칸 찾는 정도	체인별 주소 선택. 없으면 `undefined` (그림 4)
`useQueryClient()`	react-query 훅	아니요. 항상 같은 객체	공유 캐시 관리자 가져오기 (그림 5, 6)

반복이 거슬린다면: 커스텀 훅

이름이 `use`로 시작하는 함수는 안에서 다른 훅을 호출할 수 있어요. 세 줄을 하나로 묶을 수 있습니다.

```
function useDeployment() {
  const connection = useConnection()
  return {
    me: connection.address,
    addresses: ADDRESSES[connection.chainId ?? 0],
    queryClient: useQueryClient(),
  }
}

// 각 컴포넌트에서
const { me, addresses, queryClient } = useDeployment()
```

동작은 완전히 똑같아요. `useDeployment()`를 호출한 컴포넌트가 렌더링 중이니, 그 안의 `useConnection()`도 그 컴포넌트의 훅 목록에 기록되고 구독도 그 컴포넌트에 걸려요(그림 2).